

Object-Oriented Programming

Catch-Up
First Final Technical Report
DistriCine: Seventh Art Made Code

Diego Alejandro Yañes Zabala (20251020103)
Brayan Sebastian Diaz Ramirez (20251020150)
Oscar Javier Camargo Trujillo (20251020143)

Universidad Distrital Francisco José de Caldas

Faculty of Engineering
Systems Engineering Program
Teacher: Ing. Carlos Andrés Sierra, M.Sc
Bogotá D.C., Colombia
2025

Contents

1	Abstract	2
2	Introduction	2
3	Background	3
3.1	Institutional and Cultural Context	3
3.2	Fundamentals of Object-Oriented Programming	3
3.3	Object Creation Strategies and OOP Principles	3
3.4	Design and Modeling Methodologies	3
3.5	User Experience (UX) and Interface (UI) Design	4
3.6	Technical Architecture and Platform	7
3.7	Iterative Development Approach	7
4	Scope	7
4.1	Project Limits and Coverage	7
4.2	Features Included in Scope	7
4.3	Exclusions from the Current Scope	8
4.4	Assumed Technical Limitations	8
5	Main Methodology: Object-Oriented Design (OOD)	8
5.1	The Role of Objects in OOP	8
5.2	Contribution of Object-Oriented Design	8
5.3	Design and Analysis Techniques	9
5.3.1	User Stories	9
5.3.2	CRC (Class-Responsibility-Collaborator) Cards	9
5.3.3	UML Modeling	9
5.4	OOP Implementation Process	9
5.4.1	Identifying Classes and Objects	9
5.4.2	Designing Inheritance Hierarchies	9
5.4.3	Encapsulation Application	10
5.4.4	Implementation of Polymorphism	10
5.5	Implementation Code	10
5.6	Complementary Methodologies Applied	15
5.6.1	Adaptive Cascade Development	15
5.6.2	Validation Techniques	15
5.6.3	Development Strategy	15
6	Results and Next Steps	16
6.1	Results and Validation	16
6.2	Next Steps and Future Work	16
7	Conclusions	16

1 Abstract

Access to film-related cultural activities in the university environment is often limited and poorly disseminated, which restricts student participation. To address this problem, this work presents DistriCine, a web-based management application for university film clubs, designed under the principles of Object-Oriented Programming (OOP). As a result, a functional prototype has been developed in a console environment that consolidates the business logic for checking schedules, managing reservations, and viewing movie listings. DistriCine lays the technical foundations for a system that centralizes and optimizes the film-related cultural experience of the student community, demonstrating the viability of a tailor-made solution for this context at the Francisco Jose de Caldas District University.

2 Introduction

The management of cultural and recreational activities at the university, such as film clubs, has traditionally relied on manual processes that include going from classroom to classroom, posting posters or distributing flyers inviting people to participate in the different screenings that have been planned; methods that are not only time-consuming but also tend to fail to attract the interest of university students, resulting in an inefficient allocation of resources, access to, and little assistance due to the students' own planning. At the Francisco José de Caldas District University, the "DistriCine" project emerged as a digital response to these challenges, proposing a comprehensive solution to manage film club screenings across multiple university locations.

Commercial cinema management systems, while comprehensive, have significant limitations for the academic environment: their excessive complexity and cost, along with the lack of specific features such as integration with university identification or adaptation to multi-site operations, make them unviable for implementation in university film clubs.

This project adopts Object-Oriented Programming (OOP) as the fundamental paradigm, acting as the "master script" that structures the system. The cinematic metaphor extends to the design: just as in a production each role has defined responsibilities, in DistriCine, encapsulation defines the responsibilities of each class, inheritance establishes a clear hierarchy of users, and polymorphism allows different components to respond specifically to common messages.

DistriCine is a web-based cinema management system specifically designed for university environments. Our solution overcomes the limitations of existing approaches by providing a customized system that allows students to check schedules, manage reservations, and access detailed film information. The system implements role-based access control with separate interfaces for administrators and customers, ensuring both security and an intuitive user experience.

The main contributions of this work include: the design and implementation of a specialized film management system for academic institutions using OOP methodologies; the systematic application of fundamental OOP principles, including inheritance for role management and encapsulation for data integrity; and an evaluation of the system's effectiveness through functional testing and architectural validation.

3 Background

3.1 Institutional and Cultural Context

Francisco José de Caldas University, through its University Film Club initiative, seeks to strengthen cultural spaces within the institution, recognizing film as a fundamental tool for students' comprehensive development. However, the manual management of film screenings across multiple campuses created significant accessibility and efficiency challenges, creating the need for a digital solution tailored to the academic context.

3.2 Fundamentals of Object-Oriented Programming

DistriCine's design is based on the Object-Oriented Programming (OOP) paradigm, systematically applying its structural principles. Encapsulation was implemented through private attributes and public access methods in all domain classes. Inheritance was realized in the user hierarchy, where `CustomerUser` and `AdminUser` extend the abstract `User` class. Polymorphism was implemented by overriding the `showMenu()` method, allowing for differentiated behaviors based on user roles.

3.3 Object Creation Strategies and OOP Principles

Object creation in DistriCine was designed by systematically applying the principles of Object-Oriented Programming, transforming simple instantiation into a mechanism that reinforces the system architecture.

Encapsulation through specialized constructors ensures that each object is initialized in a valid and consistent state, incorporating business validation from the moment of creation. This approach ensures that entity instances like `User` and `Movie` comply with domain invariants from their inception.

Structural relationships were implemented through composition, where container objects create and manage the lifecycle of their associated components. This "has-a" relationship strategy allows for the construction of complex structures while maintaining controlled coupling and fostering cohesion within each module.

Constructor overloading provides creation interfaces tailored to different usage contexts, offering a polymorphic approach to initialization that simplifies common use cases while maintaining full specification capability.

Inheritance in the user hierarchy demonstrates the Liskov substitution principle, where subclasses can be used transparently in place of the base class, allowing role-specific attributes to be initialized while maintaining the common interface.

3.4 Design and Modeling Methodologies

The design process used the Unified Modeling Language (UML) as a standard for visualizing and documenting the system architecture. Class diagrams captured the structural relationships between entities, while sequence diagrams modeled dynamic interactions. Additionally, CRC Cards (Class-Responsibility-Collaborator) were used to identify and refine

the responsibilities of each class and their collaborations, facilitating a balanced distribution of functionality across the system.

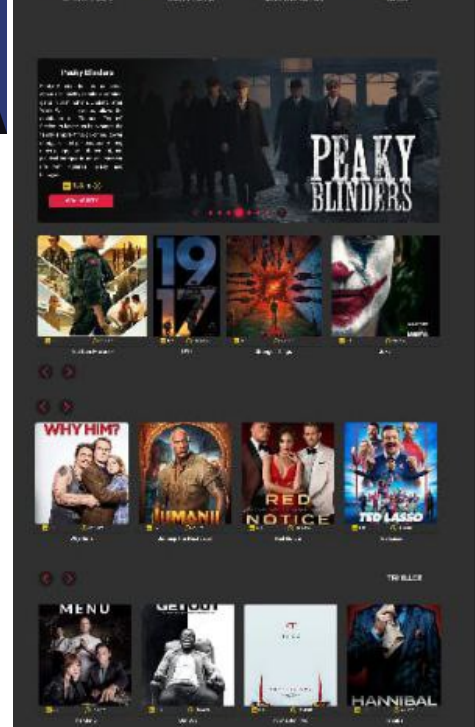
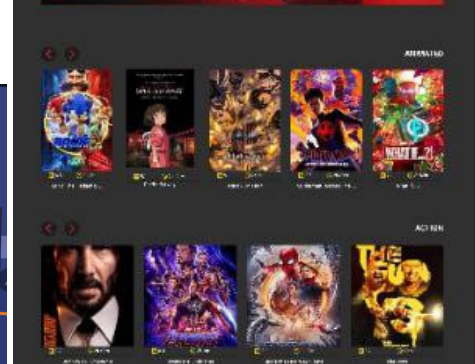
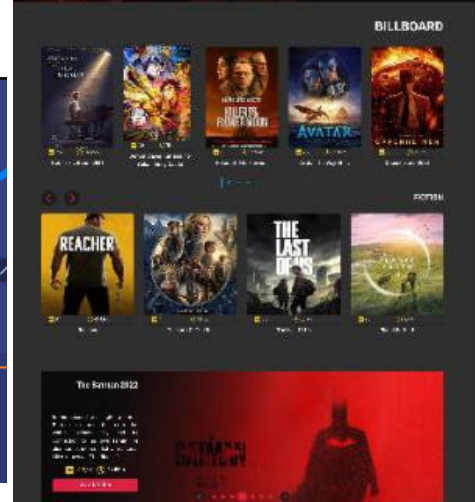
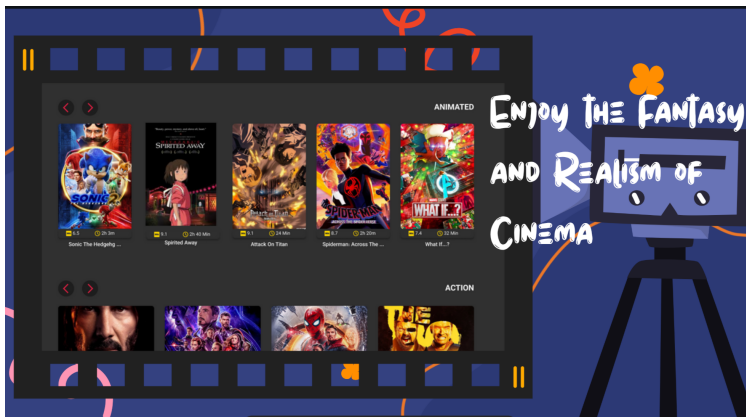
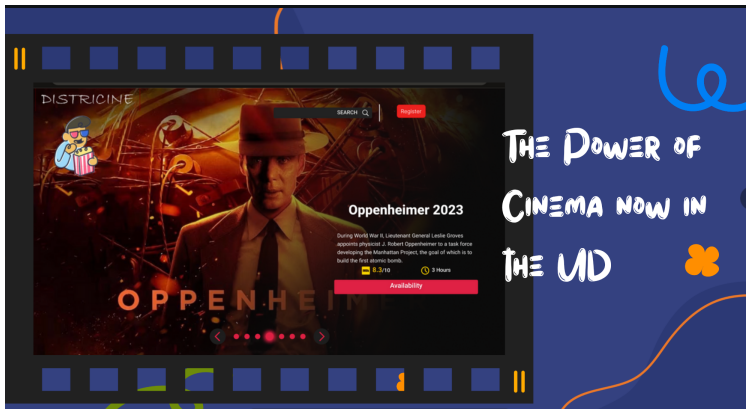
3.5 User Experience (UX) and Interface (UI) Design

The app was designed following User Experience (UX) principles focused on the specific needs of the student community, prioritizing efficiency in critical workflows such as booking tickets and checking schedules. The User Interface (UI) was planned with an emphasis on simplicity and visual consistency, using Figma to create iterative prototypes that validated usability and navigation flows.

The user interface (UI) was planned with an emphasis on simplicity and visual consistency, using Figma to create iterative prototypes that validated usability and navigation flows. The system mockups are presented below:

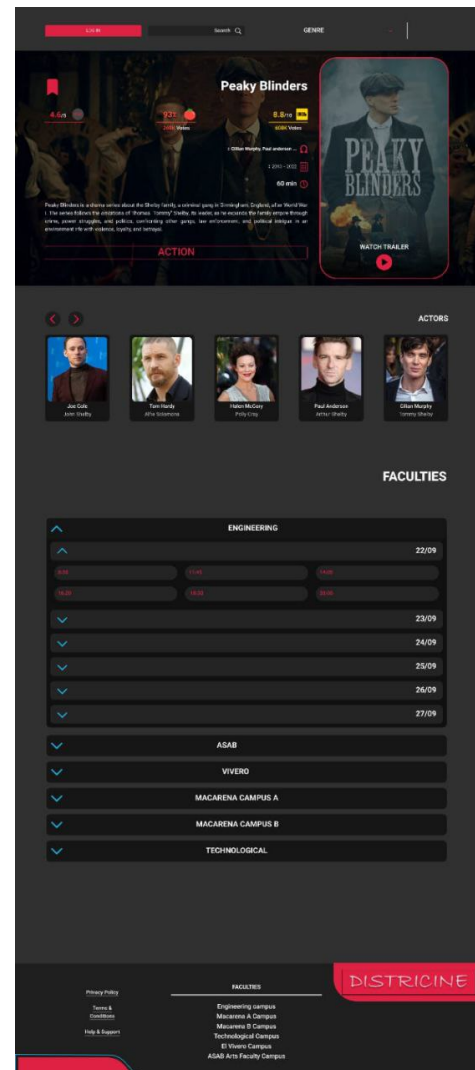
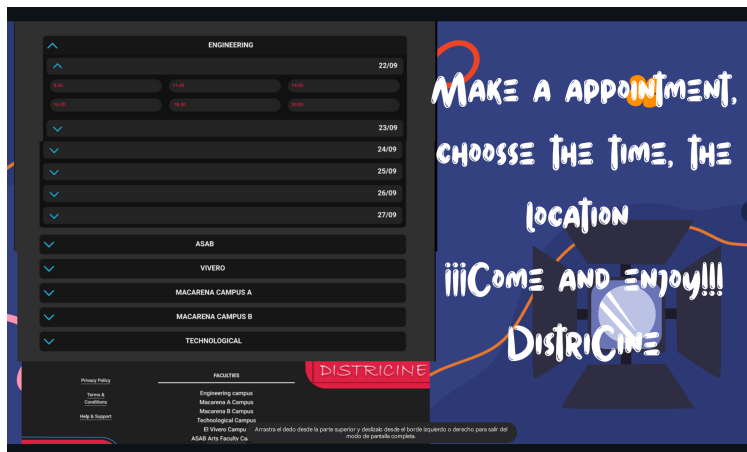
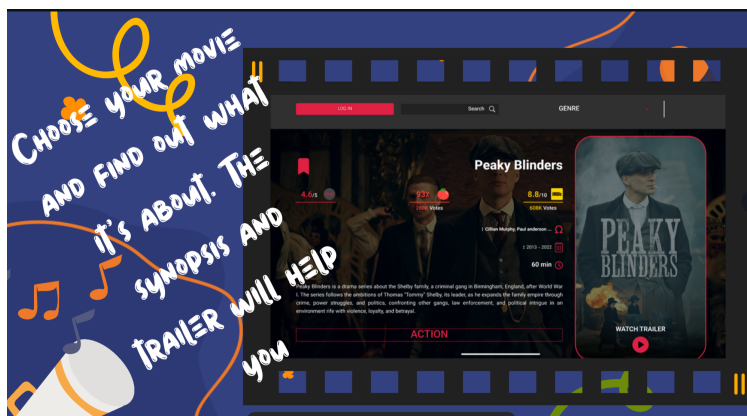
Mockup 1

The login, registration, and search options are located in the upper left corner, as this facilitates user actions. Additionally, we usually move our eyes from right to left, allowing better exploration of the program. As one of the program's main objectives is to allow the user to schedule their cinema attendance, the top banner—which covers a wide area—shows the latest release or the movie that the cinema will screen.



Mockup 2

When the user clicks on a selected movie, they are directed to this page where the movie poster and a synopsis are displayed. Regarding the rating or voting system, this feature allows the user to form an idea about the movie. The trailer provides a preview of the film, generating impact and attraction for the user. This section also shows available dates and times: here the user can see the days the film will be shown, if scheduled, along with the exact times so that the user can plan accordingly.



It's important to note that while the interface design is fully defined at the prototype level, the implementation of the graphical interface is planned for a later phase of the project. Currently, the system operates through a console, with the logic being developed for subsequent implementation.

3.6 Technical Architecture and Platform

The implementation was done in Java, taking advantage of its robustness for the object-oriented paradigm and its cross-platform portability. The current architecture includes the main classes running in a console environment, with a clear separation between the domain layer (business entities), the application layer (flow coordination), and the presentation layer (currently implemented through a text interface). This approach allows for the validation of the system's functionality, allowing for continued development and the construction of the graphical interface.

3.7 Iterative Development Approach

The project adopted an agile methodology with iterative design-implementation-validation cycles. This approach allowed for progressive refinement of the UML models and CRC Cards based on continuous feedback, ensuring that the object-oriented abstractions faithfully reflected the requirements of the university film domain. Phased development allows for a robust functional core before investing in the implementation of the graphical interface.

This theoretical and methodological foundation provides the basis for understanding the architectural and design decisions implemented in DistriCine, establishing a coherent framework that integrates sound software engineering principles with the specific needs of the university context, and explaining the current state of development of the project.

4 Scope

4.1 Project Limits and Coverage

The development of DistriCine has been limited considering time restrictions, so it has been progressed gradually as required in this type of applications at the programming level, since we are in the process of acquiring the knowledge and know-how necessary to achieve greater breadth in terms of functionality, allowing this to establish a realistic scope that guarantees the delivery of a functional and quality product. The project focuses on implementing the essential functionalities that solve the most critical needs of the university community and the need to have the Film Club in accordance with new technologies.

4.2 Features Included in Scope

The project's scope includes the complete development of a system that allows users to view available film listings, access detailed information about each film (title, genre, runtime, synopsis, and rating), check screening times, and find out which university venues will host the screenings. The system also includes a reservation module that allows students to register their interest in attending a specific screening.

The implementation covers the management of two types of users: students who use the system for inquiries and reservations, and administrators who manage the film catalog content. All business logic related to these processes is fully developed and operates in a console environment, validating the system's core workflows.

It's important to note that the graphical interface design is visualized from its development and prototyping in Figma, including login screens, movie catalog, movie details, showtime selection, and the booking process. These prototypes establish the visual and user experience foundation for a future graphical implementation.

4.3 Exclusions from the Current Scope

The system does not handle specific seat assignments within rooms, considering only general availability by function. Integration with existing university systems (such as institutional authentication or student databases) is postponed for future iterations, and for now, a separate user system is used.

4.4 Assumed Technical Limitations

The movie catalog is assumed to be static during program execution, with no real-time update capabilities. Data persistence is maintained in memory during the active session, with no permanent database implementation at this stage. Trailers and promotional material are referenced but not directly integrated into this version.

5 Main Methodology: Object-Oriented Design (OOD)

The DistriCine project was developed following the Object-Oriented Design methodology, focusing on the problem domain, progressively and systematically applying the fundamental concepts of OOP. The fundamental role of objects in this methodology was to serve as modular units that encapsulate both state (data) and behavior (operations), allowing for the direct and intuitive modeling of real-world entities. This methodological approach was based on an iterative process that combined domain analysis, structural modeling, and implementation guided by principles of encapsulation, inheritance, polymorphism, and abstraction.

5.1 The Role of Objects in OOP

In the development of DistriCine, objects played essential roles as software representations of entities in the film domain. Each object was designed as an independent entity with its own identity, state, and behavior, facilitating the modeling of concepts such as users, films, reservations, and schedules. Collaboration between objects through message sending allowed complex functionalities to be built from simple interactions, creating a system where responsibilities were distributed in a natural and coherent manner.

5.2 Contribution of Object-Oriented Design

Object-Oriented Design brought multiple benefits to the DistriCine project. It allowed the system to be organized into cohesive, loosely coupled modules, facilitating code maintenance and evolution. The ability to model real-world relationships through inheritance and composition was especially valuable for representing the user hierarchy and the associations between

movies, showtimes, and bookings. Furthermore, the object-oriented approach promoted code reuse and laid the groundwork for a scalable and extensible architecture.

5.3 Design and Analysis Techniques

5.3.1 User Stories

User stories were developed following the format "As a [role], I want [goal] for [benefit]," which allowed functional requirements to be captured from the end-user perspective. These stories served as a basis for identifying the responsibilities that should be assigned to the different classes of the system and for defining the acceptance criteria that would validate the correct functioning of each feature.

5.3.2 CRC (Class-Responsibility-Collaborator) Cards

CRC cards were used as a fundamental tool for object-oriented design, allowing for the identification and refinement of the responsibilities of each class and its collaborators. This technique facilitated the balanced distribution of functionality among classes and helped visualize relationships and dependencies within the system before proceeding with implementation, ensuring proper cohesion and controlled coupling between components.

5.3.3 UML Modeling

The Unified Modeling Language (UML) was used to visually represent the system architecture. Class diagrams captured the static structure of the system, showing attributes, methods, and relationships between classes. Sequence diagrams complemented this model by specifying the dynamic interactions between objects during the execution of the most important use cases, providing a complete view of the system's behavior.

5.4 OOP Implementation Process

5.4.1 Identifying Classes and Objects

The process began with the identification of key entities in the university film domain, identifying candidate classes such as User, Movie, Schedule, Reservation, and Theater through a thorough domain analysis. Each of these entities was analyzed to determine its essential attributes and fundamental behaviors, laying the groundwork for faithful modeling of the real-world problem.

5.4.2 Designing Inheritance Hierarchies

An inheritance hierarchy was established with the User class as the base class, from which CustomerUser and AdminUser are derived. This structure allowed for the reuse of common code while specializing behavior according to the user's role. It applied the basic OOP inheritance mechanism to share attributes and methods between related classes, demonstrating the principle of extensibility without modification.

5.4.3 Encapsulation Application

All class attributes were declared as private, providing controlled access through public getter and setter methods. This encapsulation protected data integrity and kept the internal state of objects consistent at all times, ensuring that modifications only occurred through well-defined and validated interfaces.

5.4.4 Implementation of Polymorphism

Polymorphism was implemented by overriding methods in subclasses, particularly the showMenu() method, which presents different interfaces for administrators and clients. This allowed objects of different classes to respond specifically to the same message, facilitating system extension and reducing dependencies between components.

5.5 Implementation Code

Below is the source code developed in this phase, including some classes and a functional test class.

Listing 1: Abstract User Class

```
/**
 * Abstract base class representing a generic user in the DistriCine
 * system.
 * Contains common attributes and declares abstract behavior that must
 * be
 * implemented by subclasses (eg, CustomerUser, AdminUser).
 */
public abstract class User {
    private String name;
    private String email;
    private String password;

    /**
     * Constructs a User with the specified name, email, and password.
     *
     * @param name the full name of the user
     * @param email the email address of the user (used for login)
     * @param password the account password (stored in plain text in this
     *     prototype)
     */
    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    // Getters and Setters
}
```

```

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPassword() { return password; }
public void setPassword(String password) { this.password = password;
    }

/**
 * Displays a role-specific menu to the user.
 * Must be implemented by concrete subclasses.
 */
public abstract void showMenu();
}

```

Listing 2: CustomerUser Class

```

/**
 * Represents a regular customer (student) in the DistriCine system.
 * Customers can view schedules, make reservations, and check their
 *   history.
 */
public class CustomerUser extends User {

    /**
     * Constructs a CustomerUser with the given credentials.
     *
     * @param name the customer's full name
     * @param email the customer's university email
     * @param password the account password
     */
    public CustomerUser(String name, String email, String password) {
        super(name, email, password);
    }

    /**
     * Displays the customer-specific menu options in the console.
     */
    @Override
    public void showMenu() {
        System.out.println("Customer Menu:");
        System.out.println("1. View schedule");
        System.out.println("2. Make a reservation");
        System.out.println("3. Check history");
    }
}

```

```

/**
 * Initiates the reservation process for a movie screening.
 * In this prototype, it only prints a confirmation message.
 */
public void makeReservation() {
System.out.println("Reservation in process...");
}
}

```

Listing 3: AdminUser Class

```

/**
 * Represents an administrator in the DistriCine system.
 * Administrators manage movies, schedules, and venue information.
 */
public class AdminUser extends User {

/**
 * Constructs an AdminUser with the given credentials.
 *
 * @param name the administrator's full name
 * @param email the admin's university email
 * @param password the account password
 */
public AdminUser(String name, String email, String password) {
super(name, email, password);
}

/**
 * Displays the administrator-specific menu options in the console.
 */
@Override
public void showMenu() {
System.out.println("Administrator Menu:");
System.out.println("1. Add Movie");
System.out.println("2. Edit Schedule");
System.out.println("3. Manage Venues");
}

/**
 * Simulates the addition of a new movie to the system catalog.
 * In this prototype, it only prints a success message.
 */
public void addMovie() {
System.out.println("New movie added to the catalog.");
}
}

```

Listing 4: Movie Class

```
/**
 * Represents a movie in the DistriCine catalog.
 * Stores basic metadata such as title, genre, and duration.
 */
public class Movie {
    private String title;
    private String genre;
    private double duration; // in minutes

    /**
     * Constructs a Movie with the specified attributes.
     *
     * @param title the movie's title
     * @param genre the primary genre (eg, "Sci-Fi", "Drama")
     * @param duration the runtime in minutes
     */
    public Movie(String title, String genre, double duration) {
        this.title = title;
        this.genre = genre;
        this.duration = duration;
    }

    // Getters (no setters to enforce immutability in this version)

    public String getTitle() { return title; }
    public String getGenre() { return genre; }
    public double getDuration() { return duration; }

    /**
     * Prints formatted movie information to the console.
     */
    public void showInfo() {
        System.out.println("Movie: " + title + " | Genre: " + genre + " | "
            + "Duration: " + duration + " min");
    }
}
```

Listing 5: Reservation Class

```
/**
 * Represents a user's reservation for a specific movie screening.
 * Links to Movie to a date and time.
 */
public class Reservation {
    private Movie movie;
    private String date; // format: "YYYY-MM-DD"
    private String time; // format: "HH:mm"
```

```

/**
 * Constructs a Reservation for the given movie, date, and time.
 *
 * @param movie the movie being reserved
 * @param date the screening date (eg, "2025-10-25")
 * @param time the screening time (eg, "19:00")
 */
public Reservation(Movie movie, String date, String time) {
    this.movie = movie;
    this.date = date;
    this.time = time;
}

/**
 * Confirms the reservation by printing details to the console.
 */
public void confirm() {
    System.out.println("Reservation confirmed for " + movie.getTitle() +
        " on " + date + " at " + time);
}
}

```

Listing 6: Main Test Class

```

/**
 * Test class to demonstrate the core functionality of the DistriCine
 * system.
 * Validates user roles, movie creation, and reservation workflow in
 * console mode.
 */
public class Main {
    public static void main(String[] args) {
        // Create sample users
        CustomerUser customer = new CustomerUser("Ana", "ana@udistrital.edu.
            co", "1234");
        AdminUser admin = new AdminUser("Carlos", "carlos@udistrital.edu.co",
            "admin123");

        // Display role-specific menus (polymorphism in action)
        customer.showMenu();
        admin.showMenu();

        // Create a movie and reservation
        Movie movie = new Movie("Inception", "Sci-Fi", 148);
        Reservation reservation = new Reservation(movie, "2025-10-25", "19:00
            ");
        reservation.confirm();
    }
}

```

```
// Simulate customer actions
customer.makeReservation();
}
}
```

This code constitutes the logic under development, is functional, mentioned in the project scope, validated by execution in a console environment and aligned with the OOP principles described.

5.6 Complementary Methodologies Applied

The development incorporated elements of evolutionary prototyping by creating interface prototypes in Figma and implementing a functional console implementation as the initial version of the system. This approach allowed for validation of both the user experience design and the technical architecture before committing resources to full implementations.

5.6.1 Adaptive Cascade Development

A light variant of waterfall development was followed, organized in sequential but iterative phases: Requirements → Design (UML/CRC) → Implementation (Java) → Testing (Console), allowing feedback between phases and adjustments based on discoveries during development.

5.6.2 Validation Techniques

The design was validated through iterative reviews of CRC cards and UML diagrams, ensuring that each class had well-defined and consistent responsibilities. User stories served as functional validation criteria, verifying that the modeled system could satisfy all identified requirements. Additionally, proofs of concept were conducted to validate collaborations between critical classes.

5.6.3 Development Strategy

An incremental approach was adopted, first building the core classes and then the relationships between them. Each development iteration focused on implementing and validating a specific set of features, always guided by OOP principles and the relationships identified in the modeling. This strategy allowed for a controlled evolution of the architecture, maintaining design consistency while progressively incorporating planned features.

This Object-Oriented Design-based methodology, complemented by proven modeling techniques such as CRC cards and UML, and enriched with evolutionary prototyping elements, enabled the creation of a coherent, maintainable architecture aligned with the DistriCine system requirements, while also laying the groundwork for future system extensions and enhancements.

6 Results and Next Steps

6.1 Results and Validation

The development of DistriCine has culminated in the creation of a robust and modular application core. It has been validated for:

Functionality Testing: Verification of main use cases (registration, query, reservation) in the console environment.

Architecture Validation: The OOD design has proven effective, producing cohesive, loosely coupled code, which facilitates maintenance and extension.

UX prototyping: User flows have been validated through Figma prototypes, ensuring an intuitive experience for the graphical interface launch.

Successful OOP Application: The principles of encapsulation, inheritance, polymorphism, and composition have been consistently applied, creating an architecture that accurately reflects the problem domain.

6.2 Next Steps and Future Work

The successful completion of this phase lays the foundation for the following stages of development:

1. Graphical User Interface (GUI) Implementation: Translate Figma prototypes into a real web application, using a modern framework to connect the UI with existing business logic.
2. Data Persistence Implementation: Migrate from in-memory storage to a permanent database of information.
3. Integration with University Systems: Connect DistriCine with the university's active directories or authentication systems for single sign-on.
4. Advanced Features: Incorporate seat selection, a preference-based recommendation system, and the integration of multimedia content such as trailers.
5. Microservices Architecture: Evolve the architecture toward a microservices approach to improve scalability and maintainability.

7 Conclusions

1. The Object-Oriented Design (OOD) approach enabled the construction of a software model consistent with the university's film industry domain, translating real-world concepts—such as users, movies, schedules, and reservations—into well-encapsulated classes with clear inheritance hierarchies and polymorphic behaviors. This modeling not only facilitated the implementation of business logic in the console but also established a maintainable, extensible architecture aligned with good software engineering practices.

2. The project seeks to strike a balance between functionality and technical feasibility by defining a scope focused on the film club's core needs: viewing listings, viewing schedules by location, managing reservations, and controlling access by role. Although the graphical interface and data persistence remain for later phases, the console prototype successfully validated the system's core flows, demonstrating the robustness of the logical core before investing in more complex layers.
3. DistriCine represents an innovative techno-cultural proposal for the academic environment, integrating user experience (UX) principles, UML modeling, CRC mapping, and iterative development within a university education context. Beyond its practical utility, the project serves as a pedagogical example of how to systematically apply Object-Oriented Programming to solve real-world problems, while laying the groundwork for a future evolution toward a complete web application with institutional integration and advanced features.